

```
In [1]: #!/ [추가 코드] 데이터 경로를 프로젝트 루트 기준으로 설정
from pathlib import Path

# 프로젝트 루트 또는 그 하위 폴더에서 노트북을 실행할 수 있도록 경로 탐색
PROJECT_ROOT = next(
    (path for path in (Path.cwd(), *Path.cwd().parents)
     if (path / "_01_code" / "_03_real_world_data_to_tensors").is_dir()),
    None,
)
if PROJECT_ROOT is None:
    raise FileNotFoundError("link_dl 프로젝트 폴더에서 노트북을 실행해 주세요.")

#!/ [추가 코드] enumerate() 결과에서 처음 2개와 마지막 2개만 출력하기 위한 함수
def should_display_enumerated_item(idx, total):
    return idx < 2 or idx >= total - 2
```

a_tabular_wine_data.py

```
In [2]: import csv
import os
import numpy as np
```

```
In [3]: wine_path = os.path.join(PROJECT_ROOT, "_00_data", "d_tabular-wine", "winequality-white.csv")    #! os 중립적 경로
wineq_numpy = np.loadtxt(wine_path, dtype=np.float32, delimiter=";", skiprows=1)    #! csv -> numpy array로 로드
print(wineq_numpy.dtype)
print(wineq_numpy.shape)
print(wineq_numpy)
```

```
float32
(4898, 12)
[[ 7.    0.27  0.36 ...  0.45  8.8   6.  ]
 [ 6.3   0.3   0.34 ...  0.49  9.5   6.  ]
 [ 8.1   0.28  0.4   ...  0.44 10.1  6.  ]
 ...
 [ 6.5   0.24  0.19 ...  0.46  9.4   6.  ]
 [ 5.5   0.29  0.3   ...  0.38 12.8   7.  ]
 [ 6.    0.21  0.38 ...  0.32 11.8   6.  ]]
```

```
In [4]: #!/ csv.reader()를 사용하여 첫 번째 행(컬럼 이름)을 읽어 리스트로 변환
col_list = next(csv.reader(open(wine_path), delimiter=';'))
print(col_list)
```

```
['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'alcohol', 'quality']
```

```
In [5]: import torch
```

```
In [6]: #!/ numpy array를 torch tensor로 변환
wineq = torch.from_numpy(wineq_numpy)
print(wineq.dtype)
print(wineq.shape)
print()
```

```
torch.float32
torch.Size([4898, 12])
```

```
In [7]: #!/ tabular data의 input <- 마지막 열을 제외한 모든 열
data = wineq[:, :-1]    #! slicing
print(data.dtype)
print(data.shape)    #! 마지막 target 열을 제외했으므로 (4898, 11)
print(data)
print()
```

```
torch.float32
torch.Size([4898, 11])
tensor([[ 7.0000,  0.2700,  0.3600, ...,  3.0000,  0.4500,  8.8000],
        [ 6.3000,  0.3000,  0.3400, ...,  3.3000,  0.4900,  9.5000],
        [ 8.1000,  0.2800,  0.4000, ...,  3.2600,  0.4400, 10.1000],
        ...,
        [ 6.5000,  0.2400,  0.1900, ...,  2.9900,  0.4600,  9.4000],
        [ 5.5000,  0.2900,  0.3000, ...,  3.3400,  0.3800, 12.8000],
        [ 6.0000,  0.2100,  0.3800, ...,  3.2600,  0.3200, 11.8000]])
```

```
In [8]: #!/ tabular data의 target <- 마지막 열
target = wineq[:, -1]    #! indexing (차원 축소 발생)
print(target.dtype)
print(target.shape)
```

```

print(target)
print()

torch.float32
torch.Size([4898])
tensor([6., 6., 6., ..., 6., 7., 6.])

```

```

In [9]: #! target의 값이 0~10 사이이므로 범주형 데이터임
#! one-hot encoding 하기 위해 int64로 변환
target = target.to(torch.int64)
print(target.dtype)
print(target.shape)
print(target)
print()

```

```

torch.int64
torch.Size([4898])
tensor([6, 6, 6, ..., 6, 7, 6])

```

```

In [10]: #! identity matrix를 one-hot 벡터를 만들기 위한 인덱스로 사용
eye_matrix = torch.eye(10)

#! target 텐서의 각 요소에 대해 one-hot 벡터를 생성할 수 있음
onehot_target = eye_matrix[target] #! 인덱싱을 사용하여 one-hot 벡터 생성

print(onehot_target.shape) # >>> torch.Size([4898, 10])
print(onehot_target[0])
print(onehot_target[1])
print(onehot_target[-2])
print(onehot_target)

```

```

torch.Size([4898, 10])
tensor([0., 0., 0., 0., 0., 0., 1., 0., 0., 0.])
tensor([0., 0., 0., 0., 0., 0., 1., 0., 0., 0.])
tensor([0., 0., 0., 0., 0., 0., 0., 1., 0., 0.])
tensor([[0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        ...,
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 1., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.]])

```

```

In [11]: #! tensor 연산을 통한 z-score standardization
data_mean = torch.mean(data, dim=0) #! (11,)
data_var = torch.var(data, dim=0) #! (11,)
data = (data - data_mean) / torch.sqrt(data_var) #! (4898, 11)
print(data_mean)
print(data_var)
print(data)

```

```

tensor([6.8548e+00, 2.7824e-01, 3.3419e-01, 6.3914e+00, 4.5772e-02, 3.5308e+01,
        1.3836e+02, 9.9403e-01, 3.1883e+00, 4.8985e-01, 1.0514e+01])
tensor([7.1211e-01, 1.0160e-02, 1.4646e-02, 2.5726e+01, 4.7733e-04, 2.8924e+02,
        1.8061e+03, 8.9455e-06, 2.2801e-02, 1.3025e-02, 1.5144e+00])
tensor([[ 1.7208e-01, -8.1761e-02,  2.1326e-01, ..., -1.2468e+00,
        -3.4915e-01, -1.3930e+00],
        [-6.5743e-01,  2.1587e-01,  4.7996e-02, ...,  7.3995e-01,
        1.3422e-03, -8.2419e-01],
        [ 1.4756e+00,  1.7450e-02,  5.4378e-01, ...,  4.7505e-01,
        -4.3677e-01, -3.3663e-01],
        ...,
        [-4.2043e-01, -3.7940e-01, -1.1915e+00, ..., -1.3130e+00,
        -2.6153e-01, -9.0545e-01],
        [-1.6054e+00,  1.1666e-01, -2.8253e-01, ...,  1.0049e+00,
        -9.6251e-01,  1.8574e+00],
        [-1.0129e+00, -6.7703e-01,  3.7852e-01, ...,  4.7505e-01,
        -1.4882e+00,  1.0448e+00]])

```

```

In [12]: from sklearn.model_selection import train_test_split

```

```

In [13]: print(data.shape)
print(onehot_target.shape)

#! data, onehot_target에는 학습용과 테스트용 데이터가 모두 포함되어 있음
#! train_test_split(): 데이터를 학습용(80%)과 테스트용(20%)으로 분리
X_train, X_test, y_train, y_test = train_test_split(data, onehot_target, test_size=0.2)

print(X_train.shape)
print(y_train.shape)

```

```
print(X_test.shape)
print(y_test.shape)
```

```
torch.Size([4898, 11])
torch.Size([4898, 10])
torch.Size([3918, 11])
torch.Size([3918, 10])
torch.Size([980, 11])
torch.Size([980, 10])
```

```
In [14]: #! tabular data를 (N, F) 형태의 tensor로 변환하고, target을 one-hot encoding으로 변환하는 함수 정의
def get_wine_data():
    wine_path = os.path.join(PROJECT_ROOT, "_00_data", "d_tabular-wine", "winequality-white.csv")
    wineq_numpy = np.loadtxt(wine_path, dtype=np.float32, delimiter=";", skiprows=1)

    wineq = torch.from_numpy(wineq_numpy)

    data = wineq[:, :-1] # Selects all rows and all columns except the last
    target = wineq[:, -1].to(torch.int64) # treat labels as an integer

    eye_matrix = torch.eye(10)
    onehot_target = eye_matrix[target]

    data_mean = torch.mean(data, dim=0)
    data_var = torch.var(data, dim=0)
    data = (data - data_mean) / torch.sqrt(data_var)

    X_train, X_valid, y_train, y_valid = train_test_split(data, onehot_target, test_size=0.2)

    return X_train, X_valid, y_train, y_valid
```

기술적 사항 및 고찰

- `np.loadtxt()` 를 사용하면 구분자 (`delimiter`)와 헤더 제외 행 (`skiprows`)을 지정하여 csv를 numpy array로 읽을 수 있음
- 원본 데이터 shape (4898, 12) 에서 마지막 열을 제외하면 input은 (4898, 11), 마지막 열만 선택하면 target은 (4898,) 가 됨
- `torch.from_numpy()` 는 numpy array를 Tensor로 변환하며, 원본이 `float32` 이므로 변환된 Tensor도 `torch.float32` dtype을 가짐
- one-hot encoding에서 target을 행 인덱스로 사용하려면 정수형이 필요하므로 `torch.int64` 로 변환해야 함
- `torch.eye(10)[target]` 은 각 품질값에 해당하는 단위행렬의 행을 선택하여 target을 (4898, 10) 형태의 one-hot Tensor로 변환함
- `dim=0` 으로 평균과 분산을 구하면 11개 feature별 통계량 (11,) 이 계산되고, (4898, 11) 데이터에 broadcasting되어 z-score standardization이 수행됨
- `next(csv.reader())` 를 통해 첫 번째 행의 feature 이름 목록을 가져올 수 있음
- `train_test_split()` 을 통해 input과 target의 대응 관계를 유지하면서 훈련용 데이터와 테스트용 데이터를 분리할 수 있음

b_tabular_california_housing.py

```
In [15]: # https://medium.com/analytics-vidhya/implement-linear-regression-on-boston-housing-dataset-by-pytorch-c5d29546
# https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset
import torch
from sklearn.datasets import fetch_california_housing
```

```
In [16]: housing = fetch_california_housing() #! 캘리포니아 주택 가격 데이터셋을 로드
print(housing.keys()) #! 데이터셋 정보가 딕셔너리로 저장되어 있음

#! data, target 분리가 이미 되어 있음
print(type(housing.data))
print(housing.data.dtype)
print(housing.data.shape)

print(housing.feature_names)

print(housing.target.shape)
print(housing.target_names)

dict_keys(['data', 'target', 'frame', 'target_names', 'feature_names', 'DESCR'])
<class 'numpy.ndarray'>
float64
(20640, 8)
['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']
(20640,)
['MedHouseVal']
```

```
In [17]: import numpy as np
```

```
In [18]: print(housing.data.min(), housing.data.max())

#! np.ndarray 연산을 통한 z-score standardization
```

```
data_mean = np.mean(housing.data, axis=0)
data_var = np.var(housing.data, axis=0)
data = (housing.data - data_mean) / np.sqrt(data_var)

target = housing.target

print(data.min(), data.max())
```

```
-124.35 35682.0
-2.3859923416733877 119.41910318829312
```

```
In [19]: from sklearn.model_selection import train_test_split
```

```
In [20]: #! np.ndarray도 train_test_split() 사용 가능
X_train, X_test, y_train, y_test = train_test_split(data, target, test_size=0.2)

X_train = torch.from_numpy(X_train)
X_test = torch.from_numpy(X_test)
y_train = torch.from_numpy(y_train)
y_test = torch.from_numpy(y_test)

print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
torch.Size([16512, 8])
torch.Size([16512])
torch.Size([4128, 8])
torch.Size([4128])
```

기술적 사항 및 고찰

- California Housing 데이터는 input (20640, 8) 과 연속적인 주택 가격 target (20640,) 으로 구성된 회귀 데이터임
- 회귀 문제의 target은 클래스 번호가 아니므로 one-hot encoding하지 않고 실수값을 그대로 사용함
- axis=0 으로 평균과 분산을 계산하면 sample 방향으로 집계되어 8개 feature별 통계량을 얻을 수 있음
- feature별 통계량 (8,) 은 데이터 (20640, 8) 에 broadcasting되어 각 feature를 독립적으로 표준화함
- numpy array 상태에서도 전처리와 train_test_split() 을 수행할 수 있으며, 이후 torch.from_numpy() 로 Tensor로 변환할 수 있음
- torch.from_numpy() 는 NumPy의 dtype을 유지하므로 변환 후 dtype을 확인하는 습관이 필요함

c_2d_image_data.py

```
In [21]: import os
import imageio.v2 as imageio
import torch
```

```
In [22]: #! 이미지 파일을 numpy array로 로드
img_arr = imageio.imread(os.path.join(PROJECT_ROOT, "_00_data", "a_image-dog", "bobby.jpg"))
print(type(img_arr))
print(img_arr.shape)    #! numpy array의 경우 (H, W, C)가 기본 형태
print(img_arr.dtype)
```

```
<class 'numpy.ndarray'>
(720, 1280, 3)
uint8
```

```
In [23]: #! numpy array를 torch tensor로 변환
img = torch.from_numpy(img_arr) # numpy array -> tensor
out = img.permute(2, 0, 1) #! (H, W, C) -> (C, H, W)
print(out.shape)
```

```
torch.Size([3, 720, 1280])
```

```
In [24]: #! image data를 순회하기 위해 폴더 내의 모든 파일명을 list로 가져오기
data_dir = os.path.join(PROJECT_ROOT, "_00_data", "b_image-cats")
filenames = [
    name for name in os.listdir(data_dir) if os.path.splitext(name)[-1] == '.png'
]
print(filenames)
```

```
['cat1.png', 'cat2.png', 'cat3.png']
```

```
In [25]: from PIL import Image
```

```
In [26]: #! 이미지 파일을 하나씩 읽어와서 numpy array로 변환
total_filenames = len(filenames)
for i, filename in enumerate(filenames):
    if should_display_enumerated_item(i, total_filenames):
        image = Image.open(os.path.join(data_dir, filename))
```

```

image.show()
img_arr = imageio.imread(os.path.join(data_dir, filename))
print(img_arr.shape)
print(img_arr.dtype)
elif i == 2:
    print("...")

```

```

(256, 256, 3)
uint8
(256, 256, 3)
uint8
(256, 256, 3)
uint8

```

```

In [27]: #! (N, C, H, W) 형태의 배치 텐서 생성
batch_size = 3
batch = torch.zeros(batch_size, 3, 256, 256, dtype=torch.uint8)

#! 이미지 파일을 하나씩 읽어와서 tensor로 변환 후 배치 텐서에 저장
for i, filename in enumerate(filenamees):
    img_arr = imageio.imread(os.path.join(data_dir, filename))
    img_t = torch.from_numpy(img_arr)
    img_t = img_t.permute(2, 0, 1)
    batch[i] = img_t

print(batch.shape) #! 이미지 3개를 쌓은 NCHW 형태: (3, 3, 256, 256)

torch.Size([3, 3, 256, 256])

```

```

In [28]: #! (0~255) -> (0.0~1.0) 사이로 정규화
batch = batch.float()
batch /= 255.0
print(batch.dtype)
print(batch.shape)

torch.float32
torch.Size([3, 3, 256, 256])

```

```

In [29]: n_channels = batch.shape[1]

#! 각 채널별로 z-score standardization
for c in range(n_channels):
    mean = torch.mean(batch[:, c]) #! batch[:, c] == batch[:, c, :, :]
    std = torch.std(batch[:, c])
    print(mean, std)
    batch[:, c] = (batch[:, c] - mean) / std

tensor(0.5799) tensor(0.2212)
tensor(0.4493) tensor(0.2068)
tensor(0.3554) tensor(0.1931)

```

기술적 사항 및 고찰

- 하나의 image data는 `imageio.imread()` 를 통해 (H, W, C) 형태의 numpy array로 로드할 수 있음
- PyTorch는 채널 우선 형식을 사용하므로 `Tensor.permute(2, 0, 1)` 로 축의 순서를 (H, W, C) 에서 (C, H, W) 로 변경함
- 여러 이미지 Tensor를 배치로 다루려면 각 이미지를 같은 크기로 맞추고 뒤 (N, C, H, W) 형태로 쌓아야 함
- `torch.zeros(3, 3, 256, 256)` 은 이미지 3개를 저장할 수 있는 NCHW 형식의 빈 batch Tensor를 생성함
- 이미지의 uint8 픽셀값을 float32 로 변환하고 255로 나누면 값의 범위가 0~255 에서 0.0~1.0 으로 스케일링됨
- 채널마다 평균과 표준편차가 다르므로 `batch[:, c]` 를 사용하여 RGB 채널별로 z-score standardization을 수행함
- iterable은 `enumerate()` 를 통해 인덱스와 값을 함께 순회할 수 있음

d_bikes_sharing_data.py

```

In [30]: import os
import numpy as np
import torch

```

```

In [31]: #! torch tensor 출력 옵션 설정
#! edgeitems: 앞뒤로 출력할 item 수
#! threshold: 출력할 item 수의 임계값, 이 값보다 많으면 생략 표시(...)를 사용
#! linewidth: 한 줄에 출력할 최대 문자 수
torch.set_printoptions(edgeitems=2, threshold=10, linewidth=100)

```

```

In [32]: bikes_path = os.path.join(PROJECT_ROOT, "_00_data", "e_time-series-bike-sharing-dataset", "hour-fixed.csv")

#! numpy.loadtxt()를 사용하여 csv 파일을 로드
bikes_numpy = np.loadtxt(
    fname=bikes_path,
    dtype=np.float32,
    delimiter=",",

```

```

skiprows=1, # 1행은 feature 이름이므로 건너뛰
converters={
    1: lambda x: float(x[8:10]) #! 날짜 형식에서 day만 float로 변환 (2011-01-07 --> 07 --> 7.0)
}
)
bikes = torch.from_numpy(bikes_numpy).to(torch.float)
print(bikes)
print(bikes.shape)

tensor([[1.0000e+00, 1.0000e+00, ..., 1.3000e+01, 1.6000e+01],
        [2.0000e+00, 1.0000e+00, ..., 3.2000e+01, 4.0000e+01],
        ...,
        [1.7378e+04, 3.1000e+01, ..., 4.8000e+01, 6.1000e+01],
        [1.7379e+04, 3.1000e+01, ..., 3.7000e+01, 4.9000e+01]])
torch.Size([17520, 17])

```

```

In [33]: #! 원본 데이터는 시간 단위로 측정된 equally spaced time series data
#! 하루(24시간) 단위로 데이터를 묶어 3차원 텐서로 변환할 수 있음
daily_bikes = bikes.view(-1, 24, bikes.shape[1]) #! (17520, 17) -> (730, 24, 17)
print(daily_bikes.shape) # >>> torch.Size([730, 24, 17])

torch.Size([730, 24, 17])

```

```

In [34]: #! data와 target을 분리
daily_bikes_data = daily_bikes[:, :, :-1]
daily_bikes_target = daily_bikes[:, :, -1].unsqueeze(dim=-1) #! -1은 마지막 차원을 추가하는 의미

print(daily_bikes_data.shape) #! 730일, 하루 24시간, input feature 16개
print(daily_bikes_target.shape) #! target의 마지막 차원을 유지한 (730, 24, 1)

torch.Size([730, 24, 16])
torch.Size([730, 24, 1])

```

```

In [35]: #! 24시간 단위 데이터에서 첫 날의 데이터를 가져옴
first_day_data = daily_bikes_data[0]
print(first_day_data.shape)

torch.Size([24, 16])

```

```

In [36]: # Weather situation: 1: clear, 2: mist, 3: light rain/snow, 4: heavy rain/snow
#! 범주형 데이터를 one-hot encoding
print(first_day_data[:, 9])
print(first_day_data[:, 9].shape, first_day_data[:, 9].dtype)
eye_matrix = torch.eye(4)
print(eye_matrix)

weather_onehot = eye_matrix[first_day_data[:, 9].to(torch.int64) - 1]
print(weather_onehot.shape)
print(weather_onehot)

tensor([1., 1., ..., 2., 2.])
torch.Size([24]) torch.float32
tensor([[1., 0., 0., 0.],
        [0., 1., 0., 0.],
        [0., 0., 1., 0.],
        [0., 0., 0., 1.]])
torch.Size([24, 4])
tensor([[1., 0., 0., 0.],
        [1., 0., 0., 0.],
        ...,
        [0., 1., 0., 0.],
        [0., 1., 0., 0.]])

```

```

In [37]: #! 첫날의 weather situation에 대한 one-hot encoding을 feature로 추가
first_day_data_torch = torch.cat(tensors=(first_day_data, weather_onehot), dim=1)
print(first_day_data_torch.shape)
print(first_day_data_torch[:, 16:])

torch.Size([24, 20])
tensor([[1., 0., 0., 0.],
        [1., 0., 0., 0.],
        ...,
        [0., 1., 0., 0.],
        [0., 1., 0., 0.]])

```

```

In [38]: #! 모든 날의 weather situation에 대한 one-hot encoding을 feature로 추가
day_data_torch_list = []

for daily_idx in range(daily_bikes_data.shape[0]): # range(730)
    day = daily_bikes_data[daily_idx] # day.shape: [24, 16]
    weather_onehot = eye_matrix[day[:, 9].to(torch.int64) - 1]
    day_data_torch = torch.cat(tensors=(day, weather_onehot), dim=1) # day_data_torch.shape: [24, 20]
    day_data_torch_list.append(day_data_torch)

print(len(day_data_torch_list))

```

```
daily_bikes_data = torch.stack(day_data_torch_list, dim=0)
print(daily_bikes_data.shape)
```

```
730
torch.Size([730, 24, 20])
```

```
In [39]: #! 불필요 컬럼을 슬라이싱과 torch.cat()를 사용하여 제거
#! 0번 컬럼(instant)는 id이므로 drop
#! 9번 컬럼(whethersit)은 one-hot encoding으로 대체되므로 drop
print(daily_bikes_data[:, :, :9].shape, daily_bikes_data[:, :, 10:].shape)
daily_bikes_data = torch.cat(
    [daily_bikes_data[:, :, 1:9], daily_bikes_data[:, :, 10:]], dim=2
)
print(daily_bikes_data.shape)

torch.Size([730, 24, 9]) torch.Size([730, 24, 10])
torch.Size([730, 24, 18])
```

```
In [40]: #! 'temp' feature를 평균 0, 표준편차 1이 되도록 z-score standardization
temperatures = daily_bikes_data[:, :, 8]
daily_bikes_data[:, :, 8] = (daily_bikes_data[:, :, 8] - torch.mean(temperatures)) / torch.std(temperatures)
```

기술적 사항 및 고찰

- 시간 단위 데이터 (17520, 17) 을 view(-1, 24, 17) 로 변환하면 24시간씩 묶인 날짜 단위 Tensor (730, 24, 17) 이 됨
- view() 에서 첫 번째 차원을 -1 로 지정하면 전체 원소 수가 유지되도록 PyTorch가 날짜 수 730을 자동 계산함
- 마지막 열을 target으로 분리한 뒤 unsqueeze(dim=-1) 을 적용하면 target shape이 (730, 24) 에서 (730, 24, 1) 로 변함
- 날씨의 원본 범주값은 1~4이고, one-hot encoding된 4개 feature의 각 값은 0 또는 1임
- (24, 16) 데이터와 (24, 4) one-hot Tensor를 dim=1 로 연결하면 하루 데이터가 (24, 20) 이 됨
- 730일의 Tensor를 torch.stack(..., dim=0) 으로 쌓으면 (730, 24, 20) 형태가 됨
- 식별자 instant 와 one-hot encoding으로 대체된 weathersit 열을 제거하면 최종 feature shape은 (730, 24, 18) 이 됨
- temp feature는 전체 평균을 빼고 표준편차로 나누어 z-score standardization함

j_linear_regression_dataset_dataloader.py

```
In [41]: import torch
from torch.utils.data import Dataset, DataLoader, random_split
```

```
In [42]: #! Dataset 클래스를 상속받아 LinearRegressionDataset 클래스 정의
class LinearRegressionDataset(Dataset):
    def __init__(self, N=50, m=-3, b=2, *args, **kwargs):
        # N: number of samples, e.g. 50
        # m: slope
        # b: offset
        super().__init__(*args, **kwargs)

        self.x = torch.rand(N, 2)
        self.noise = torch.rand(N) * 0.2 #! 0.0~0.2 사이의 노이즈
        self.m = m
        self.b = b
        self.y = (
            torch.sum(self.x * self.m, dim=1) + self.b + self.noise
        ).unsqueeze(-1)

    #! len() 함수로 객체의 길이를 반환할 때 호출되는 메서드
    def __len__(self):
        return len(self.x)

    #! 객체를 인덱싱할 때 호출되는 메서드
    def __getitem__(self, idx):
        return self.x[idx], self.y[idx] #! (input, target) 튜플 반환

    #! print() 함수로 객체를 출력할 때 호출되는 메서드
    def __str__(self):
        str = "Data Size: {0}, Input Shape: {1}, Target Shape: {2}".format(
            len(self.x), self.x.shape, self.y.shape
        )
        return str
```

```
In [43]: linear_regression_dataset = LinearRegressionDataset()
print(linear_regression_dataset) #! __str__() 메서드 호출
```

```
Data Size: 50, Input Shape: torch.Size([50, 2]), Target Shape: torch.Size([50, 1])
```

```
In [44]: #! Dataset 객체는 iterable이므로 enumerate()를 사용하여 순회 가능
total_samples = len(linear_regression_dataset)
for idx, sample in enumerate(linear_regression_dataset):
    input, target = sample #! __getitem__() 메서드 호출
    if should_display_enumerated_item(idx, total_samples):
```

```

print("{0} - {1}: {2}".format(idx, input, target))
elif idx == 2:
    print("...")

```

```

0 - tensor([0.6651, 0.8005]): tensor([-2.2885])
1 - tensor([0.6492, 0.2243]): tensor([-0.4592])
...
48 - tensor([0.8410, 0.8091]): tensor([-2.7757])
49 - tensor([0.9999, 0.6163]): tensor([-2.7390])

```

```

In [45]: #!/ random_split() 함수를 사용하여 Dataset 객체를 학습용, 검증용, 테스트용으로 분리
train_dataset, validation_dataset, test_dataset = random_split(linear_regression_dataset, [0.7, 0.2, 0.1])
print(len(train_dataset), len(validation_dataset), len(test_dataset))

```

```

35 10 5

```

```

In [46]: train_data_loader = DataLoader(
    dataset=train_dataset,
    batch_size=4,
    shuffle=True #! 데이터를 랜덤하게 섞음
)

```

```

In [47]: #!/ DataLoader 객체는 iterable이므로 enumerate()를 사용하여 순회 가능
#!/ batch_size=4이므로 한 번에 4개의 샘플이 튜플로 묶여서 반환됨
total_batches = len(train_data_loader)
for idx, batch in enumerate(train_data_loader):
    input, target = batch
    if should_display_enumerated_item(idx, total_batches):
        print("{0} - {1}: {2}".format(idx, input, target))
    elif idx == 2:
        print("...")

```

```

0 - tensor([[0.7245, 0.2422],
           [0.3998, 0.3294],
           [0.7196, 0.5712],
           [0.9024, 0.0070]]): tensor([[ -0.7483],
           [-0.1823],
           [-1.7843],
           [-0.6359]])
1 - tensor([[0.2741, 0.9676],
           [0.9860, 0.5093],
           [0.6996, 0.6333],
           [0.3756, 0.3424]]): tensor([[ -1.6592],
           [-2.4240],
           [-1.9892],
           [-0.0160]])
...
7 - tensor([[0.4996, 0.1201],
           [0.5767, 0.7830],
           [0.1774, 0.9691],
           [0.2713, 0.0235]]): tensor([[ 0.1928],
           [-2.0381],
           [-1.3660],
           [ 1.2483]])
8 - tensor([[0.7357, 0.9191],
           [0.1353, 0.5156],
           [0.5243, 0.9650]]): tensor([[ -2.8452],
           [ 0.0994],
           [-2.4453]])

```

기술적 사항 및 고찰

- Dataset 객체는 input과 target을 저장하고, `__len__()` 과 `__getitem__()` 을 통해 sample의 개수와 개별 sample을 제공함
- input `self.x` 의 shape은 `(N, 2)` 이며, `torch.sum(..., dim=1)` 은 각 sample의 두 feature만 합산하여 `(N,)` 결과를 만듦
- `unsqueeze(-1)` 을 적용하면 회귀 target shape이 `(N,)` 에서 `(N, 1)` 로 변하여 input의 sample 차원과 일치함
- `dim` 을 지정하지 않고 `torch.sum()` 을 호출하면 모든 sample과 feature가 하나의 값으로 합쳐지므로 샘플별 target을 만들 수 없음
- DataLoader 객체는 Dataset 의 여러 sample을 batch 차원으로 묶어 전달함
- `random_split()` 에 `[0.7, 0.2, 0.1]` 비율을 전달하면 50개 sample이 train 35개, validation 10개, test 5개로 나뉨
- train data 35개를 `batch_size=4` 로 묶으면 마지막 batch에는 3개의 sample이 포함됨
- Dataset 과 DataLoader 는 모두 iterable이므로 `enumerate()` 를 통해 인덱스와 데이터를 함께 순회할 수 있음

k_2d_image_dataset_dataloader.py

```
In [48]: import os

import torch
from PIL import Image
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision import transforms
```

```
In [49]: class DogCat2DImageDataset(Dataset):
    def __init__(self):
        #! 이미지 전처리(transform) 정의
        self.image_transforms = transforms.Compose([
            transforms.Resize(size=(256, 256)), #! 이미지 크기 조정
            transforms.ToTensor() #! tensor 변환 및 (C, H, W) permutation 수행
        ])
        #! 이미지 로드
        dogs_dir = os.path.join(PROJECT_ROOT, "_00_data", "a_image-dog")
        cats_dir = os.path.join(PROJECT_ROOT, "_00_data", "b_image-cats")

        image_lst = [
            Image.open(os.path.join(dogs_dir, "bobby.jpg")), # (1280, 720, 3)
            Image.open(os.path.join(cats_dir, "cat1.png")), # (256, 256, 3)
            Image.open(os.path.join(cats_dir, "cat2.png")), # (256, 256, 3)
            Image.open(os.path.join(cats_dir, "cat3.png")) # (256, 256, 3)
        ]

        #! 이미지를 (C, H, W) 형태의 tensor로 변환
        image_lst = [self.image_transforms(img) for img in image_lst]
        print(image_lst[0].shape, image_lst[1].shape, image_lst[2].shape, image_lst[3].shape)

        #! input: (N, C, H, W) 형태의 배치 텐서
        self.images = torch.stack(image_lst, dim=0) #! (4, 3, 256, 256)

        #! target: (N, 1) 형태의 배치 텐서 (0: "dog", 1: "cat")
        self.image_labels = torch.tensor([[0], [1], [1], [1]]) #! (4, 1)

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        return self.images[idx], self.image_labels[idx]

    def __str__(self):
        str = "Data Size: {0}, Input Shape: {1}, Target Shape: {2}".format(
            len(self.images), self.images.shape, self.image_labels.shape
        )
        return str
```

```
In [50]: dog_cat_2d_image_dataset = DogCat2DImageDataset()
print(dog_cat_2d_image_dataset)

torch.Size([3, 256, 256]) torch.Size([3, 256, 256]) torch.Size([3, 256, 256]) torch.Size([3, 256, 256])
Data Size: 4, Input Shape: torch.Size([4, 3, 256, 256]), Target Shape: torch.Size([4, 1])
```

```
In [51]: total_samples = len(dog_cat_2d_image_dataset)
for idx, sample in enumerate(dog_cat_2d_image_dataset):
    input, target = sample
    if should_display_enumerated_item(idx, total_samples):
        print("{0} - {1}: {2}".format(idx, input.shape, target))
    elif idx == 2:
        print("...")

0 - torch.Size([3, 256, 256]): tensor([0])
1 - torch.Size([3, 256, 256]): tensor([1])
2 - torch.Size([3, 256, 256]): tensor([1])
3 - torch.Size([3, 256, 256]): tensor([1])
```

```
In [52]: #! random_split() 함수를 사용하여 Dataset을 학습용, 테스트용으로 분리
train_dataset, test_dataset = random_split(dog_cat_2d_image_dataset, [0.7, 0.3])
print(len(train_dataset), len(test_dataset))

3 1
```

```
In [53]: train_data_loader = DataLoader(
    dataset=train_dataset,
    batch_size=2,
    shuffle=True
)
```

```
In [54]: total_batches = len(train_data_loader)
for idx, batch in enumerate(train_data_loader):
    input, target = batch #! batch_size=2인데 train_dataset의 크기가 3이므로 마지막 배치에서는 batch_size=1이 됨
    if should_display_enumerated_item(idx, total_batches):
```

```
print("{0} - {1}: {2}".format(idx, input.shape, target))
elif idx == 2:
    print("...")
```

```
0 - torch.Size([2, 3, 256, 256]): tensor([[0],
      [1]])
1 - torch.Size([1, 3, 256, 256]): tensor([[1]])
```

기술적 사항 및 고찰

- `torchvision.transforms.Compose()` 를 사용하면 여러 이미지 전처리 연산을 정의된 순서대로 적용할 수 있음
- `transforms.Resize((256, 256))` 는 서로 다른 크기의 이미지를 동일한 높이와 너비로 맞춤
- `transforms.ToTensor()` 는 PIL 이미지를 Tensor로 변환하면서 축을 (H, W, C) 에서 (C, H, W) 로 바꾸고 픽셀값을 0.0~1.0 범위로 스케일링함
- 이미지 4개를 `torch.stack(..., dim=0)` 으로 쌓으면 batch 차원이 추가되어 input shape이 (4, 3, 256, 256) 이 됨
- 개별 sample의 input shape은 (3, 256, 256), target shape은 (1,) 임
- 4개 sample을 [0.7, 0.3] 비율로 분리하면 남은 sample이 앞쪽 split에 배분되어 train 3개, test 1개가 됨
- train sample 3개를 `batch_size=2` 로 묶으면 첫 batch는 2개, 마지막 batch는 1개가 됨

l_wine_dataset_dataloader.py

```
In [55]: import os
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader, random_split
```

```
In [56]: class WineDataset(Dataset):
    def __init__(self):
        wine_path = os.path.join(PROJECT_ROOT, "_00_data", "d_tabular-wine", "winequality-white.csv")
        wineq_numpy = np.loadtxt(wine_path, dtype=np.float32, delimiter=";", skiprows=1)
        wineq = torch.from_numpy(wineq_numpy)

        #! data 전처리: z-score standardization
        data = wineq[:, :-1] # Selects all rows and all columns except the last
        data_mean = torch.mean(data, dim=0)
        data_var = torch.var(data, dim=0)
        self.data = (data - data_mean) / torch.sqrt(data_var)

        #! target 전처리: one-hot encoding
        target = wineq[:, -1].to(torch.int64) # treat labels as an integer
        eye_matrix = torch.eye(10)
        self.target = eye_matrix[target]

        assert len(self.data) == len(self.target)

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        wine_feature = self.data[idx]
        wine_target = self.target[idx]
        return wine_feature, wine_target

    def __str__(self):
        str = "Data Size: {0}, Input Shape: {1}, Target Shape: {2}".format(
            len(self.data), self.data.shape, self.target.shape
        )
        return str
```

```
In [57]: wine_dataset = WineDataset()

print(wine_dataset)
```

```
Data Size: 4898, Input Shape: torch.Size([4898, 11]), Target Shape: torch.Size([4898, 10])
```

```
In [58]: total_samples = len(wine_dataset)
for idx, sample in enumerate(wine_dataset):
    input, target = sample
    if should_display_enumerated_item(idx, total_samples):
        print("{0} - {1}: {2}".format(idx, input.shape, target.shape))
    elif idx == 2:
        print("...")

0 - torch.Size([11]): torch.Size([10])
1 - torch.Size([11]): torch.Size([10])
...
4896 - torch.Size([11]): torch.Size([10])
4897 - torch.Size([11]): torch.Size([10])
```

```
In [59]: #! random_split() 함수를 사용하여 Dataset을 학습용, 검증용, 테스트용으로 분리
train_dataset, validation_dataset, test_dataset = random_split(wine_dataset, [0.7, 0.2, 0.1])

print(len(train_dataset), len(validation_dataset), len(test_dataset))
```

3429 980 489

```
In [60]: train_data_loader = DataLoader(
    dataset=train_dataset,
    batch_size=32,
    shuffle=True,
    drop_last=True #! 마지막 배치에서 batch_size보다 작은 샘플이 남으면 버림
)
```

```
In [61]: total_batches = len(train_data_loader)
for idx, batch in enumerate(train_data_loader):
    input, target = batch
    if should_display_enumerated_item(idx, total_batches):
        print("{0} - {1}: {2}".format(idx, input.shape, target.shape))
    elif idx == 2:
        print("...")
```

```
0 - torch.Size([32, 11]): torch.Size([32, 10])
1 - torch.Size([32, 11]): torch.Size([32, 10])
...
105 - torch.Size([32, 11]): torch.Size([32, 10])
106 - torch.Size([32, 11]): torch.Size([32, 10])
```

기술적 사항 및 고찰

- Wine 데이터의 로드, z-score standardization, one-hot encoding을 `Dataset` 내부에 캡슐화하면 같은 전처리가 모든 sample에 일관되게 적용됨
- `assert len(self.data) == len(self.target)` 을 통해 input과 target의 sample 수가 같은지 확인할 수 있음
- `__getitem__()` 이 반환하는 개별 sample의 input shape은 `(11,)` , one-hot target shape은 `(10,)` 임
- `DataLoader` 가 32개 sample을 묶으면 input shape은 `(32, 11)` , target shape은 `(32, 10)` 이 됨
- `shuffle=True` 는 epoch마다 train sample의 순서를 섞어 특정 순서에 과도하게 의존하는 것을 줄임
- `drop_last=True` 는 마지막 불완전한 batch를 제외하여 모든 train batch의 크기를 32로 일정하게 유지함

m_california_housing_dataset_dataloader.py

```
In [62]: import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader, random_split
```

```
In [63]: class CaliforniaHousingDataset(Dataset):
    def __init__(self):
        from sklearn.datasets import fetch_california_housing
        housing = fetch_california_housing()
        #! data, target 전처리
        data_mean = np.mean(housing.data, axis=0)
        data_var = np.var(housing.data, axis=0)
        self.data = torch.tensor((housing.data - data_mean) / np.sqrt(data_var), dtype=torch.float32)
        self.target = torch.tensor(housing.target, dtype=torch.float32).unsqueeze(dim=-1)

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        sample_data = self.data[idx]
        sample_target = self.target[idx]
        return sample_data, sample_target

    def __str__(self):
        str = "Data Size: {0}, Input Shape: {1}, Target Shape: {2}".format(
            len(self.data), self.data.shape, self.target.shape
        )
        return str
```

```
In [64]: california_housing_dataset = CaliforniaHousingDataset()

print(california_housing_dataset)
```

Data Size: 20640, Input Shape: torch.Size([20640, 8]), Target Shape: torch.Size([20640, 1])

```
In [65]: total_samples = len(california_housing_dataset)
for idx, sample in enumerate(california_housing_dataset):
    input, target = sample
    if should_display_enumerated_item(idx, total_samples):
```

```
print("{0} - {1}: {2}".format(idx, input.shape, target.shape))
elif idx == 2:
    print("...")
```

```
0 - torch.Size([8]): torch.Size([1])
1 - torch.Size([8]): torch.Size([1])
...
20638 - torch.Size([8]): torch.Size([1])
20639 - torch.Size([8]): torch.Size([1])
```

```
In [66]: train_dataset, validation_dataset, test_dataset = random_split(california_housing_dataset, [0.7, 0.2, 0.1])

print(len(train_dataset), len(validation_dataset), len(test_dataset))

14448 4128 2064
```

```
In [67]: train_data_loader = DataLoader(
    dataset=train_dataset,
    batch_size=32,
    shuffle=True,
    drop_last=True
)
```

```
In [68]: total_batches = len(train_data_loader)
for idx, batch in enumerate(train_data_loader):
    input, target = batch
    if should_display_enumerated_item(idx, total_batches):
        print("{0} - {1}: {2}".format(idx, input.shape, target.shape))
    elif idx == 2:
        print("...")

0 - torch.Size([32, 8]): torch.Size([32, 1])
1 - torch.Size([32, 8]): torch.Size([32, 1])
...
449 - torch.Size([32, 8]): torch.Size([32, 1])
450 - torch.Size([32, 8]): torch.Size([32, 1])
```

기술적 사항 및 고찰

- California Housing 데이터의 input shape은 (20640, 8) 이고 8개의 주택 관련 feature를 가짐
- 회귀 문제의 target은 범주형 label이 아니므로 one-hot encoding하지 않고 실수형 Tensor로 사용함
- NumPy에서 계산한 표준화 결과를 dtype=torch.float32 로 지정하여 모델 학습에서 일반적으로 사용하는 dtype으로 변환함
- unsqueeze(dim=-1) 을 통해 target을 (20640,) 에서 (20640, 1) 형태로 변환할 수 있음
- Dataset 은 input (8,) 과 target (1,) 형태의 개별 sample을 반환함
- DataLoader 는 여러 sample을 묶어 input (32, 8) , target (32, 1) 형태의 batch를 반환함
- drop_last=True 를 사용하면 마지막 불완전한 batch가 제거되므로 학습에 사용되는 전체 데이터 수가 줄어든 수 있음

n_time_series_dataset_dataloader.py

```
In [69]: import os
from pathlib import Path

import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader, random_split

import sys
```

```
In [70]: BASE_PATH = str(PROJECT_ROOT)
sys.path.append(BASE_PATH)
```

```
In [71]: class BikesDataset(Dataset):
    def __init__(self, train=True, test_days=1):
        self.train = train
        self.test_days = test_days

        bikes_path = os.path.join(BASE_PATH, "_00_data", "e_time-series-bike-sharing-dataset", "hour-fixed.csv")

        bikes_numpy = np.loadtxt(
            fname=bikes_path, dtype=np.float32, delimiter=",", skiprows=1,
            converters={
                1: lambda x: float(x[8:10]) # 2011-01-07 --> 07 --> 7
            }
        )
        bikes = torch.from_numpy(bikes_numpy) #! (17520, 17)

        #! 24시간 단위로 데이터를 묶어 3차원 텐서로 변환
        daily_bikes = bikes.view(-1, 24, bikes.shape[1]) #! (730, 24, 17)
```

```

#! data와 target을 분리
self.daily_bikes_target = daily_bikes[:, :, -1].unsqueeze(dim=-1) #! (730, 24, 1)
self.daily_bikes_data = daily_bikes[:, :, :-1] #! (730, 24, 16)

#! 날씨 정보 one-hot encoding 후 feature로 추가
eye_matrix = torch.eye(4)
day_data_torch_list = []

for daily_idx in range(self.daily_bikes_data.shape[0]): # range(730)
    day = self.daily_bikes_data[daily_idx] #! (24, 16)
    weather_onehot = eye_matrix[day[:, 9].to(torch.int64) - 1]
    day_data_torch = torch.cat(tensors=(day, weather_onehot), dim=1) #! (24, 20)
    day_data_torch_list.append(day_data_torch)

self.daily_bikes_data = torch.stack(day_data_torch_list, dim=0) #! (730, 24, 20)

#! 원본 날씨 정보 drop
self.daily_bikes_data = torch.cat(
    [self.daily_bikes_data[:, :, :9], self.daily_bikes_data[:, :, 10:]], dim=2
) #! (730, 24, 19)

#! 시계열 데이터이므로 나중 날짜를 테스트용으로 남겨둘 수 있음
#! test_days를 기준으로 데이터를 학습용과 테스트용으로 분리
total_length = len(self.daily_bikes_data)
self.train_bikes_data = self.daily_bikes_data[:total_length - test_days]
self.train_bikes_targets = self.daily_bikes_target[:total_length - test_days]
train_temperatures = self.train_bikes_data[:, :, 9]
train_temperatures_mean = torch.mean(train_temperatures)
train_temperatures_std = torch.std(train_temperatures)
self.train_bikes_data[:, :, 9] = (self.train_bikes_data[:, :, 9] - train_temperatures_mean) / train_temperatures_std

assert len(self.train_bikes_data) == len(self.train_bikes_targets)

self.test_bikes_data = self.daily_bikes_data[-test_days:]
self.test_bikes_targets = self.daily_bikes_target[-test_days:]
self.test_bikes_data[:, :, 9] = (self.test_bikes_data[:, :, 9] - train_temperatures_mean) / train_temperatures_std

assert len(self.test_bikes_data) == len(self.test_bikes_targets)

def __len__(self):
    return len(self.train_bikes_data) if self.train is True else len(self.test_bikes_data)

def __getitem__(self, idx):
    bike_feature = self.train_bikes_data[idx] if self.train is True else self.test_bikes_data[idx]
    bike_target = self.train_bikes_targets[idx] if self.train is True else self.test_bikes_targets[idx]
    return bike_feature, bike_target

def __str__(self):
    if self.train is True:
        str = "Data Size: {0}, Input Shape: {1}, Target Shape: {2}".format(
            len(self.train_bikes_data), self.train_bikes_data.shape, self.train_bikes_targets.shape
        )
    else:
        str = "Data Size: {0}, Input Shape: {1}, Target Shape: {2}".format(
            len(self.test_bikes_data), self.test_bikes_data.shape, self.test_bikes_targets.shape
        )
    return str

```

```

In [72]: #! 학습용 데이터셋: test_days 만큼의 날짜가 제외됨
train_bikes_dataset = BikesDataset(train=True, test_days=1)
print(train_bikes_dataset) #! (729, 24, 19), (729, 24, 1)

```

Data Size: 729, Input Shape: torch.Size([729, 24, 19]), Target Shape: torch.Size([729, 24, 1])

```

In [73]: #! 학습용, 검증용 데이터셋으로 분리
train_dataset, validation_dataset = random_split(train_bikes_dataset, [0.8, 0.2])
print(len(train_dataset), len(validation_dataset)) #! (584, 145)

```

584 145

```

In [74]: print("[TRAIN]")
total_samples = len(train_dataset)
for idx, sample in enumerate(train_dataset):
    input, target = sample
    if should_display_enumerated_item(idx, total_samples):
        print("{0} - {1}: {2}".format(idx, input.shape, target.shape))
    elif idx == 2:
        print("...")

```

```
[TRAIN]
0 - torch.Size([24, 19]): torch.Size([24, 1])
1 - torch.Size([24, 19]): torch.Size([24, 1])
...
582 - torch.Size([24, 19]): torch.Size([24, 1])
583 - torch.Size([24, 19]): torch.Size([24, 1])
```

```
In [75]: #! 원활한 학습을 위해 shuffle, drop_last 옵션 사용
train_data_loader = DataLoader(
    dataset=train_dataset,
    batch_size=32,
    shuffle=True,
    drop_last=True
)
```

```
In [76]: total_batches = len(train_data_loader)
for idx, batch in enumerate(train_data_loader):
    input, target = batch
    if should_display_enumerated_item(idx, total_batches):
        print("{0} - {1}: {2}".format(idx, input.shape, target.shape))
    elif idx == 2:
        print("...")

0 - torch.Size([32, 24, 19]): torch.Size([32, 24, 1])
1 - torch.Size([32, 24, 19]): torch.Size([32, 24, 1])
...
16 - torch.Size([32, 24, 19]): torch.Size([32, 24, 1])
17 - torch.Size([32, 24, 19]): torch.Size([32, 24, 1])
1 - torch.Size([32, 24, 19]): torch.Size([32, 24, 1])
...
16 - torch.Size([32, 24, 19]): torch.Size([32, 24, 1])
17 - torch.Size([32, 24, 19]): torch.Size([32, 24, 1])
```

```
In [77]: print("[VALIDATION]")
total_samples = len(validation_dataset)
for idx, sample in enumerate(validation_dataset):
    input, target = sample
    if should_display_enumerated_item(idx, total_samples):
        print("{0} - {1}: {2}".format(idx, input.shape, target.shape))
    elif idx == 2:
        print("...")

[VALIDATION]
0 - torch.Size([24, 19]): torch.Size([24, 1])
1 - torch.Size([24, 19]): torch.Size([24, 1])
...
143 - torch.Size([24, 19]): torch.Size([24, 1])
144 - torch.Size([24, 19]): torch.Size([24, 1])
```

```
In [78]: #! 학습에 사용되지 않으므로 shuffle, drop_last를 고려하지 않아도 됨
validation_data_loader = DataLoader(
    dataset=validation_dataset,
    batch_size=32
)
```

```
In [79]: total_batches = len(validation_data_loader)
for idx, batch in enumerate(validation_data_loader):
    input, target = batch
    if should_display_enumerated_item(idx, total_batches):
        print("{0} - {1}: {2}".format(idx, input.shape, target.shape))
    elif idx == 2:
        print("...")

0 - torch.Size([32, 24, 19]): torch.Size([32, 24, 1])
1 - torch.Size([32, 24, 19]): torch.Size([32, 24, 1])
...
3 - torch.Size([32, 24, 19]): torch.Size([32, 24, 1])
4 - torch.Size([17, 24, 19]): torch.Size([17, 24, 1])
```

```
In [80]: #! 테스트용 데이터셋: test_days 만큼의 날짜만 포함됨
test_dataset = BikesDataset(train=False, test_days=1)
print(test_dataset) #! (1, 24, 19), (1, 24, 1)
```

Data Size: 1, Input Shape: torch.Size([1, 24, 19]), Target Shape: torch.Size([1, 24, 1])

```
In [81]: print("[TEST]")
total_samples = len(test_dataset)
for idx, sample in enumerate(test_dataset):
    input, target = sample
    if should_display_enumerated_item(idx, total_samples):
        print("{0} - {1}: {2}".format(idx, input.shape, target.shape))
    elif idx == 2:
        print("...")
```

```
[TEST]
0 - torch.Size([24, 19]): torch.Size([24, 1])
```

```
In [82]: #! 학습에 사용되지 않으므로 shuffle, drop_last를 고려하지 않아도 됨
test_data_loader = DataLoader(
    dataset=test_dataset,
    batch_size=len(test_dataset) #! 전체 데이터를 한 번에 로드
)
```

```
In [83]: total_batches = len(test_data_loader)
for idx, batch in enumerate(test_data_loader):
    input, target = batch
    if should_display_enumerated_item(idx, total_batches):
        print("{0} - {1}: {2}".format(idx, input.shape, target.shape))
    elif idx == 2:
        print("...")
```

```
0 - torch.Size([1, 24, 19]): torch.Size([1, 24, 1])
```

기술적 사항 및 고찰

- 미래 날짜에 해당하는 test data는 `random_split()` 하지 않고 시간 순서를 유지하여 마지막 `test_days` 일에서 분리함
- test 이전의 과거 데이터는 모델 선택을 위해 `random_split()` 으로 train과 validation으로 나눔
- 날씨값을 4개의 one-hot feature로 추가하고 원본 `weathersit` 열을 제거하면 input shape은 `(730, 24, 19)` 가 됨
- 이 예제는 원본 코드의 구조에 따라 식별자 `instant` 열을 유지하므로, 앞의 `d_bikes_sharing_data.py` 보다 feature가 하나 더 많음
- test data에는 미래 데이터의 통계량을 사용하지 않고 train data에서 계산한 평균과 표준편차를 적용하여 정보 누출을 방지함
- train `DataLoader` 는 학습 순서를 섞기 위해 `shuffle=True` 를 사용하고 batch 크기를 일정하게 유지하기 위해 `drop_last=True` 를 사용함
- validation과 test에서는 학습을 수행하지 않으므로 순서를 섞거나 마지막 batch를 제거할 필요가 없음

숙제 후기

tabular data와 image data를 Tensor로 변환하는 과정은 상당히 다르다는 것을 알게 되었다. tabular data는 하나의 csv 파일을 읽은 뒤 마지막 열을 기준으로 input과 target을 비교적 쉽게 분리할 수 있었다. 반면 image data는 여러 이미지 파일을 각각 읽고, shape을 `(H, W, C)` 에서 PyTorch가 사용하는 `(C, H, W)` 로 변경한 다음 batch 차원으로 쌓아야 했다. 또한 이미지에는 target 정보가 포함되어 있지 않으므로 직접 label을 만들어 주는 과정도 필요했다.

이번 과제를 통해 Tensor 연산에서는 값뿐만 아니라 shape과 dtype을 계속 확인하는 것이 중요하다는 것을 느꼈다. 특히 indexing, slicing, `permute()`, `unsqueeze()`, `stack()`, `cat()` 을 사용할 때마다 Tensor의 차원이 어떻게 변하는지 이해해야 했다. 범주형 데이터를 one-hot encoding하면 하나의 범주값이 여러 개의 0과 1로 구성된 feature로 변환된다는 것도 알게 되었다.

`Dataset` 은 개별 input과 target을 관리하고, `DataLoader` 는 여러 sample을 batch 단위로 묶어서 전달한다는 차이도 이해할 수 있었다. 시계열 데이터는 일반 데이터와 달리 미래 데이터를 test data로 남겨 두어야 하며, test data를 표준화할 때도 train data에서 구한 평균과 표준편차를 사용해야 정보 누출을 막을 수 있다는 점이 인상적이었다.

처음에는 실행 결과의 Tensor가 너무 크고 shape도 계속 달라져 코드를 따라가기 어려웠다. 하지만 각 연산 전후의 shape을 직접 출력하고 비교하면서 PyTorch가 데이터를 처리하는 방식을 조금 더 구체적으로 이해할 수 있었다. 출력 결과가 지나치게 길어지는 부분을 정리하는 것도 생각보다 번거로웠지만, 필요한 결과를 명확하게 보여 주는 방법을 고민해 볼 수 있었다.